

מכינה תקשוב · יחידה 2

משתנים וטיפוסי נתונים

איך שומרים מידע בתוכנית — לא רק מדפיסים אותו, אלא גם זוכרים אותו.

תזכורת מהיחידה הקודמת

מה למדנו ביחידה 1?

- הפקודה `print()` מדפיסה על המסך
- טקסט חייב **מרכאות** — "כך"
- מספר **בלי** מרכאות Python — יודעת שאפשר לחשב איתו
- Python מריצה שורה אחרי שורה, מלמעלה למטה

טיפ. היום נלמד איך לשמור מידע בתוכנית, כדי שנוכל להשתמש בו שוב ושוב — לא רק להדפיס אותו פעם אחת.

מטרות השיעור

מה נלמד היום?

- מה זה **משתנה**, ואיך יוצרים אחד
- איך משתמשים בערך ששמור במשתנה, שוב ושוב
- ארבעה סוגי נתונים בסיסיים: **מספר שלם**, **מספר עשרוני**, **טקסט** ו**אמת/שקר**
- איך בודקים בעצמכם מאיזה טיפוס משתנה מסוים

לפני שמתחילים

למה בכלל צריך משתנים?

בלי משתנים, כל פעם שרוצים להשתמש בערך מסוים צריך להקליד אותו מחדש. אם הערך משתנה — צריך לחפש ולתקן אותו בכל מקום שהוא מופיע.
עם משתנים, **שומרים את הערך פעם אחת**, נותנים לו שם — ומשם והלאה פשוט קוראים לו בשם הזה, שוב ושוב, גם אם הערך עצמו משתנה בהמשך.

טיפ. זו לא רק נוחות — זו הדרך שבה תוכניות "זוכרות" מידע לאורך זמן, במקום רק להדפיס משהו פעם אחת ולשכוח אותו.

מושג מרכזי

מה זה בעצם משתנה?

דמינו **קופסה עם תווית**. אתם כותבים על התווית שם (*age* לשמל), ומכניסים לתוך הקופסה ערך (17 לשמל).

מעכשיו, בכל פעם שתמצאו את הערך הזה — פשוט קוראים לקופסה בשמה. אין צורך לזכור את הערך בעצמכם, **Python זוכרת בשבילכם**.

אפשר גם לפתוח את הקופסה ולהחליף את מה שבפנים בכל רגע — זה בדיוק למה קוראים לזה **משתנה**.
בחירת שמות

איך בוחרים שם למשתנה?

תנו למשתנה שם שמתאר **מה הוא שומר** — כך הקוד שלכם יהיה קריא, גם לכם עצמכם בעוד שבוע, וגם לכל מי שיקרא אותו.

שמות פחות מומלצים

x, a1, temp

שמות טובים

age, student_name, price

טיפ. ביחידה 4 נלמד את כל הכללים הפורמליים לשמות משתנים. כרגע — פשוט תבחרו שמות הגיוניים.

כמה כללים כבר עכשיו

מה אסור בשם משתנה?

את הרשימה המלאה של כללי השמות נלמד ביחידה 4 — אבל שלושה כללים בסיסיים שווה להכיר כבר עכשיו:

print

אסור להשתמש במילה ששמורה ל-
Python עצמה, כמו print או if

age1

שם משתנה לא יכול להתחיל
בספרה — age1 כן תקין

student name

אסור רווח בשם משתנה — ישתמשו
בין תחתון: student_name

תחביר

איך יוצרים משתנה

כותבים שם, סימן שווה, ואת הערך שרוצים לשמור. זה נקרא "הצבה: (assignment)"

```
age = 17
name = "דנה"

print(name)
```

פלט:

דנה

תרגול מידי

נסו עכשיו, לפני שממשיכים

קחו שתי דקות ונסו בעצמכם:

1. [קל] צרו משתנה בשם `city` עם עיר המגורים שלכם, והדפיסו אותו.
2. [קל] צרו משתנה בשם `favorite_number` עם מספר לבחירתכם, והדפיסו אותו.

מודל חשיבה

כמה קופסאות בבת אחת

בתוכנית אחת אפשר להחזיק הרבה משתנים במקביל — כל אחד עם השם והערך שלו, בלי שהם מתערבבים: כל קופסה עצמאית לגמרי — שינוי באחת לא משפיע על האחרות.

טיפ שמות

שם משתנה מתאר תוכן, לא טיפוס

מתחילים לפעמים נוטים לתת שמות כמו `str_name` או `num_age` — מוסיפים את סוג הנתון לשם עצמו. זה מיותר, ולפעמים אפילו מטעה.

`str_name, num_age`

מיותר, ומסוכן: מה קורה אם משנים את הטיפוס בהמשך?

`name, age`

פשוט ותיאורי — מספיק כדי להבין מה שמור בפנים

מוסכמת שמות

— `snake_case` סגנון הכתיבה של Python

כל שפת תכנות פיתחה נוהג משלה לשמות משתנים מרובי-מילים. ב-Python המוסכמה היא `snake_case` — אותיות קטנות, עם קו תחתון בין מילים:

`studentName`

זו מוסכמה אחרת, (camelCase) נפוצה בשפות כמו JavaScript ו-Java

`student_name`

כך כותבים ב-Python — קווים תחתונים, אותיות קטנות

טיפ. שתי הצורות "עובדות" טכנית ב-Python אבל מתכנתי Python נצמדים ל-`snake_case` כדי שקוד יראה אחיד לכולם.

שימו לב

מראות שוב קובעות הכל

בהינתן `name = "דנה"` — יש הבדל ענק בין:

print(name)

מדפיס **דנה** — כי בלי מרכאות Python, מחפשת משתנה בשם הזה ומדפיסה את הערך שבתוכו

print("name")

מדפיס את המילה **name** ממש — כי "name" בתוך מרכאות הוא טקסט רגיל

שימו לב. זו אחת הטעויות הכי נפוצות של מתחילים. תמיד שאלו את עצמכם: יש כאן מרכאות או לא?

מלכודת נוספת

מה קורה אם המשתנה לא קיים?

מה קורה אם מנסים להדפיס משתנה שמעולם לא הוגדר? למשל `print(username)` בלי ש-`username` הוגדר קודם:

הודעת שגיאה:

```
NameError: name 'username' is not defined
```

שימו לב — זו שגיאה **שונה** מ `SyntaxError` — שראינו ביחידה 1. **NameError** אומר: "התחביר תקין, אבל אין לי מושג על איזה משתנה אתה מדבר."

שימו לב. הגורם הכי נפוץ לזה: טעות הקלדה בשם המשתנה, או שימוש במשתנה לפני שהוא הוגדר.

טיפ בונוס

type() — איך לבדוק בעצמכם

מתלבטים מאיזה טיפוס משתנה מסוים? יש כלי מובנה שעונה על זה מיד:

```
age = 17
print(type(age))
```

פלט:

```
<class 'int'>
```

ארבעה סוגים בסיסיים

טיפוסי נתונים ב-Python-**— float מספר עשרוני**

לדוגמה: 1.75, 3.14 — מכיל נקודה

— int מספר שלם

לדוגמה: 2024, -3, 7 — בלי נקודה עשרונית

— bool אמת / שקר

רק שני ערכים אפשריים: True או False

— str (טקסט) מחזור

לדוגמה: "שלום" — תמיד בתוך מרכאות

בדקו הבנה

נחשו את הטיפוס

לכל אחד מהערכים הבאים — ענו: int, float, str או bool?

1. [קל] 42 — איזה טיפוס?

2. [קל] "42" — איזה טיפוס? (שימו לב למרכאות!)

3. [קל] 3.0 — איזה טיפוס?

4. [קל] False — איזה טיפוס?

דוגמה: ארבעת הטיפוסים

כל טיפוס בתוכנית אחת

נגדיר משתנה מכל סוג, ונדפיס את כולם:

```

grade = 95          # int
average = 87.5      # float
student = "נועה"   # str
passed = True        # bool

print(student, grade, average, passed)

```

פלט:

```

87.5 95 נועה True

```

· עוד תרגול() type

type() על כל הטיפוסים

אותו כלי בדיקה, על כל אחד מארבעת הטיפוסים:

```
print(type(87.5))
print(type("נועה"))
print(type(True))
```

פלט:

```
<class 'float'>
<class 'str'>
<class 'bool'>
```

דוגמה יהעתקת ערך

משתנה יכול לקבל ערך ממשתנה אחר

אפשר גם להעביר ערך ששמור כבר במשתנה אחד, אל משתנה חדש Python — פשוט מעתיקה את מה שנמצא שם:

```
first_name = "דנה"
display_name = first_name

print(display_name)
```

פלט:

דנה

עוד על bool

True ו-False — לא כמו שנראה

שני הערכים היחידים מטיפוס bool הם True ו-False — ויש להם כלל חשוב: תמיד **אות ראשונה גדולה**, בלי מרכאות.

true / "True"

לא תקין — אות קטנה תיתן שגיאה, ומרכאות יהפכו את זה לטקסט רגיל

True

תקין — כך Python מזהה את זה כערך בוליאני

ערכי bool יהיו קריטיים בהמשך הקורס — כשנלמד **תנאים**, (if) נשתמש בהם כל הזמן כדי להחליט "כן" או "לא". משתנה יכול להשתנות

למה קוראים לזה משתנה

אפשר לתת למשתנה ערך חדש בכל רגע — הערך הישן פשוט נשכח:

```
score = 10
print(score)

score = 25
print(score)
```

פלט:

```
10
25
```

דוגמה · הקלדה דינמית

אותו משתנה, טיפוס אחר לגמרי

אפשר אפילו לתת למשתנה ערך מטיפוס שונה לגמרי מזה שהיה לו קודם — בלי הכרזה מיוחדת:

```
x = 5
print(x)

x = "חמש"
print(x)
```

פלט:

```
5
חמש
```

תרגול מייד

נסו עכשיו bool: בפעולה

קחו דקה ונסו:

1. [קלן] צרו משתנה בשם `is_student` עם הערך `True`, והדפיסו אותו יחד עם `type()` שלו.

השוואה

Python מול Java: הקלדה דינמית

בשפות רבות (Java ומכ) חייבים להצהיר **מראש** מאיזה טיפוס יהיה כל משתנה — ואי אפשר לשנות את הטיפוס שלו אחר כך:

(טיטס הדלקה) Java

```
int age = 17;
```

(תימניד הדלקה) Python

```
age = 17
```

ב Python-לא מצהירים על הטיפוס בכלל Python — מזהה אותו לבד מהערך, ואפילו מאפשרת להחליף טיפוס בהמשך (כמו שראינו בדוגמה הקודמת). זו אחת הסיבות שהיא כה נוחה למתחילים.

בנוס

הצבה מרובה בשורה אחת

טריק נחמד: אפשר ליצור כמה משתנים בשורה אחת בלבד, אם מפרידים בפסיקים משני הצדדים:

```
a, b, c = 1, 2, 3
print(a, b, c)
```

פלט:

```
1 2 3
```

מוסכמת קבועים

משתנים שלא אמורים להשתנות

לפעמים יש ערך שלא אמור להשתנות במהלך התוכנית — למשל מספר מקסימלי של ניסיונות, או ערך קבוע כמו פאי. ב Python-אין מנגנון שאוכף את זה, אבל יש **מוסכמת שמות**:

MAX_ATTEMPTS = 3

שם משתנה כולו באותיות גדולות מאותת לכל מי שקורא: "זה אמור להישאר קבוע" — אל תשני אותי בהמשך"

דוגמה מלאה

החלפת ערכים בין שני משתנים

רוצים להחליף ביניהם — בלי לאבד אף אחד מהערכים? צריך משתנה עזר שליש, שישמור זמנית אחד מהם:

```
a = 5
b = 9

temp = a    # בצד a שומרים את
a = b       # b מקבל את a
b = temp    # a מקבל את b
            # המקורי

print(a, b)
```

פלט:

9 5

שימו לב

מלכודות נפוצות

שימו לב. שימוש במשתנה לפני שהוגדר Python — מריצה מלמעלה למטה. אם `print(score)` מופיע לפני `score = 10` — שגיאה.

שימו לב. רגישות לאותיות רישיות — Age ו-age הם שני משתנים שונים לגמרי, לא אותו דבר.

שימו לב. לשכוח מרכאות בטעות — `print(name)` לעומת `print("name")` נותנים תוצאות שונות לגמרי.

דוגמה מסכמת

פרופיל אישי — כל מה שלמדנו יחד

משתנים מכל הטיפוסים, בתוכנית אחת קטנה:

```
name = "עומר"
age = 16
height = 1.72
is_student = True

print("שם:", name)
print("גיל:", age)
```

פלט:

```
שם: עומר
גיל: 16
```

תרגול · חלק א'

משימות — רמה קלה

השתמשו רק במשתנים ובפקודת `print()` שלמדנו עד כה.

1. [קל] צרו משתנה בשם `name` עם השם שלכם, והדפיסו אותו.
2. [קל] צרו שלושה משתנים (`age`, `name`) מספר, `city` — והדפיסו כל אחד בשורה נפרדת.
3. [קל] בהינתן `age = 17` — מה ההבדל בין `print("age")` ל-`print(age)`? נסו את שתיהן והסבירו.
4. [קל] איתור באג: `Age = 17` ואז `print(age)` — לא עובד. מה הבעיה, ואיך מתקנים?

תרגול · חלק ב'

משימות — רמה בינונית

כאן נדרש שילוב של כמה טיפוסים נתונים יחד.

1. [בינוני] צרו ארבעה משתנים — אחד מכל טיפוס (`int`, `float`, `str`, `bool`) — והדפיסו כל אחד עם תווית, למשל: מספר שלם: 7.
2. [בינוני] נחשו לפני שמריצים: `print(x), x = 10, print(x), x = 5` — מה יודפס בפעם השנייה, ולמה?
3. [בינוני] איתור באג: `print(score):` מופיע לפני `score = 90` — מה השגיאה, ואיך מסדרים נכון?
4. [בינוני] צרו משתנה `price` עם מחיר עשירי (`float`), שנו אותו פעמיים (למשל בעקבות הנחה), והדפיסו את הערך אחרי כל שינוי.
5. [בינוני] צרו משתנה `x` עם ערך מספרי, השתמשו ב-`type(x)` כדי לבדוק את הטיפוס שלו, והדפיסו את התוצאה.

תרגול · חלק ב' (המשך)

עוד משימות —רמה בינונית

עכשיו עם קבועים, הצבה מרובה, ואיתור שגיאות. `NameError`

1. [בינוני] צרו קבוע בשם `MAX_SCORE` עם הערך 100, והדפיסו משפט שמשתמש בו, למשל: `הציון המקסימלי: 100`.
2. [בינוני] צרו שלושה משתנים בשורה אחת בעזרת הצבה מרובה: `x, y, z = 10, 20, 30` — והדפיסו את כולם.
3. [בינוני] איתור באג: `print(total)` מודפס לפני ש-`total` הוגדר בכלל. איזו שגיאה תקבל, ואיך מתקנים?

תרגול חלק ג'

משימות —רמה מאתגרת

המשימות האלה דורשות חשיבה על סדר הביצוע של השורות.

1. [מאתגר] בנו "פרופיל אישי" עם לפחות 5 משתנים מטיפוסים שונים (`bool`-כ/ה, `float`-כ הבוג, ליג, סש) והדפיסו את כולם בשורת פלט אחת מסודרת בעזרת פסיקים.
2. [מאתגר] החלפת ערכים: בהינתן `a = 5` ו-`b = 9`, החליפו ביניהם את הערכים בעזרת משתנה עזר שלישי לשמל (`temp`, כך שבסוף `a` יהיה 9 ו-`b` יהיה 5). הדפיסו את שניהם כדי להוכיח שזה עבד.
3. [מאתגר] בקוד הבא יש שלוש שגיאות שונות — מצאו ותקנו את כולן:


```
print(score)
score = 88
city = "Tel Aviv"
```

תרגול חלק ג' (המשך)

עוד משימות —רמה מאתגרת

שילוב של כמה כלים חדשים יחד.

1. [מאתגר] בעזרת הצבה מרובה בשורה אחת, צרו שלושה משתנים `a`, `b`, `c` עם הערכים 1, 2, 3, ואז החליפו בין `a` ל-`c` בעזרת משתנה עזר, כך שבסוף `a = 3` ו-`c = 1`. הדפיסו את כל השלושה.
2. [מאתגר] בנו "לוח בקרה" עם שני קבועים (`MIN_SCORE` ו-`MAX_ATTEMPTS` לשמל) ושלושה משתנים רגילים, והדפיסו הכל בפורמט מסודר שמבדיל חזותית בין קבועים למשתנים רגילים.

תרגול חלק ד'

משימות —מאתגר במיוחד

תרגילי מעקב (`trace`) ואיתור באגים ברמה גבוהה יותר.

1. [מאתגר] טבלת מעקב (`trace table`) בהינתן הקוד הבא, בלי להריץ — כתבו על דף מה מודפס בכל אחד משני ה-`print(x, y)`:


```
print(x, y)
y = x
x = 10
print(x, y)
x = 3
```

 אחר כך הריצו ובדקו את עצמכם.
2. [מאתגר] איתור באג משולש: מצאו ותקנו שלוש שגיאות שונות (רישיות, סדר הגדרה, ומרכאות חסרות) בקוד קצר שהמורה יציג על הלוח.
3. [מאתגר] בנו "רשומת תלמיד/ה לסמסטר" עם 5+ משתנים ממגוון טיפוסים. הדפיסו את המצב ה"התחלתי", ואז שני לפחות שני ערכים (למשל ציון ומצב לימודים) והדפיסו שוב את המצב "אחרי העדכון" — כך שרואים בבירור מה השתנה.

תרגול חלק ה'

אתגר בונוס

לא לה שרועים להתאמן עוד קצת.

1. [מאתגר] טבלת מעקב (trace table): בהינתן $x = x + 0, y = x, x = 2$ (בלי לחשב עדיין — רק לשער), מה יודפס ב-`print(x, y)`? נמקו לפני שמריצים.
2. [מאתגר] כתבו תוכנית עם משתנה יחיד שמשתנה טיפוס פעמיים ברצף (bool ל-`str` או `int`-מ), עם `print` אחרי כל שינוי כדי להראות את כל השלבים.

בדיקה עצמית

שאלות חזרה מהירות

בלי לגלול אחורה — כמה מהן אתם זוכרים?

1. [קל] מה ההבדל בין `print(name)` ל-`print("name")`?
2. [קל] מה ההבדל בין `SyntaxError` ל-`NameError`?
3. [בינוני] מה זה `snake_case` ולמה משתמשים בו?
4. [בינוני] מה ההבדל בין הקלדה דינמית (`Python`) להקלדה סטטית (`Java`)?

לפני שסוגרים

חמישה דברים לזכור מהיום

type()
הכלי שבודק מאיזה טיפוס משתנה מסוים

int / float / str / bool
ארבעת טיפוסים הנתונים הבסיסיים

name = value
כך יוצרים משתנה — שם, סימן שווה, וערך

NameError
קורה כשמשמשים במשתנה שלא הוגדר

snake_case
מוסכמת השמות של Python — אותיות קטנות וקו תחתון

עכשיו התוכניות שלכם זוכרות מידע

בשיעור הבא: אופרטורים וביטויים — נלמד לבצע חישובים אמיתיים עם המשתנים שלנו.